

Báo cáo bài tập EX14.2 Implementation of Recurrent Neural Networks from Scratch

1. One-hot tương đương với một Embedding cụ thể

Mỗi vector one-hot của từ thứ i thực chất là một vector có số 1 ở vị trí i và 0 ở nơi khác. Khi bạn nhân ma trận đầu vào X (dạng one-hot) với ma trận trọng số W , kết quả thu được chính là hàng thứ i của ma trận W .

Vì vậy, có thể coi mỗi hàng của ma trận trọng số đầu tiên chính là một "embedding vector" riêng biệt cho từng đối tượng. One-hot chỉ là cách chúng ta "lấy" vector đó ra.

[illegible]

2. Điều gì xảy ra nếu không "gọt" gradient (Gradient Clipping)?

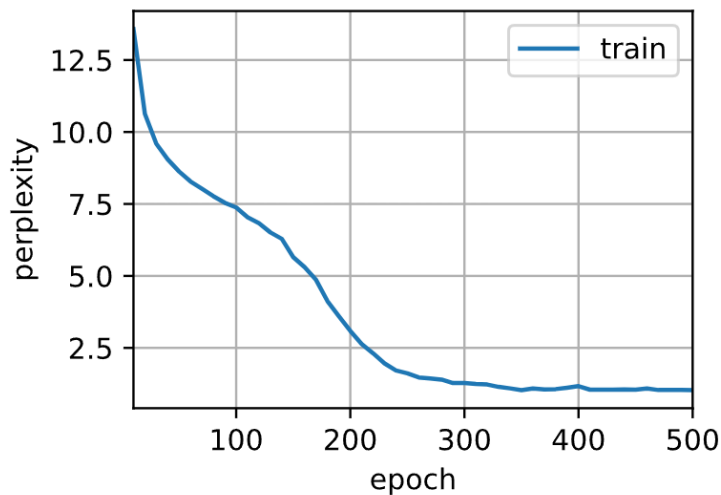
Như chúng ta đã thảo luận, nếu không có bước `grad_clipping`:

Gradient sẽ bị bùng nổ (\$exploding\$ \$gradients\$).

Trên biểu đồ: Giá trị Loss/Perplexity sẽ nhảy loạn xạ hoặc biến thành NaN (Not a Number).

Mô hình sẽ hoàn toàn không học được gì vì các bước nhảy trọng số quá lớn làm hỏng toàn bộ các giá trị đã học trước đó.

```
perplexity 1.0, 58369.2 tokens/sec on cpu
time traveller for so it will be convenient to speak of himwas e
traveller fimbneer atlwill sher the lantht bubsies bathem t
```



3. Thay đổi dự đoán: Lấy mẫu (Sampling) thay vì lấy xác suất cao nhất (Argmax)

Hiện tại hàm `predict_ch8` dùng `argmax` để chọn ký tự có xác suất cao nhất.

Nếu dùng `Argmax`: Máy tính sẽ viết rất "an toàn", thường lặp đi lặp lại những cấu trúc đơn giản (nhưng đôi khi bị lặp từ vô tận).

Nếu dùng `Sampling` (Lấy mẫu ngẫu nhiên dựa trên phân phối xác suất): * Văn bản tạo ra sẽ đa dạng, sáng tạo và "người" hơn.

4. Thay đổi hàm kích hoạt bằng `ReLU`

Thông thường RNN dùng `tanh` vì nó giới hạn giá trị trong khoảng $(-1, 1)$

Nếu dùng `ReLU`, giá trị lớp ẩn có thể tăng lên vô hạn.

Hệ quả: Kết hợp với đặc tính hồi quy của RNN, `ReLU` rất dễ gây ra bùng nổ giá trị (activation explosion), khiến việc huấn luyện trở nên cực kỳ khó khăn nếu không kiểm soát tốt tốc độ học và gọt gradient thật chặt.